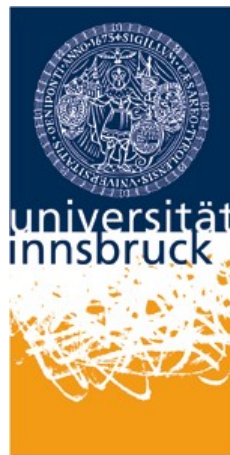


Leopold-Franzens Universität Innsbruck  
Faculty of Mathematics, Computer Science and  
Physics

Department of Computer Science



## Bachelor's Thesis

submitted for the degree of

## Bachelor of Science

## Symbolic Regression for Surrogate Modeling - A Systematic Study

von

Tobias Albrecht  
(Matr.-Nr.: 12206456)

Submission Date: **TODO: Add Submission Date**  
Supervisor: Prof. Philipp Zech, PhD

## Eidesstattliche Erklärung

*Ich erkläre hiermit an Eides statt durch meine eigenhändige Unterschrift, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Alle Stellen, die wörtlich oder inhaltlich den angegebenen Quellen entnommen wurden, sind als solche kenntlich gemacht.*

*Ich erkläre mich mit der Archivierung der vorliegenden Bachelorarbeit einverstanden.*

Ort und Datum: \_\_\_\_\_

Unterschrift: \_\_\_\_\_

## Disclaimer on the Use of AI

*TODO: Add Disclaimer on the Use of AI*

# Abstract

Surrogate models are often used in scientific and engineering applications to approximate computationally expensive simulations. However, many of the currently used surrogate models are black box models which provide almost no insight into the process of their actual learning. Symbolic Regression (SR) addresses this limitation by discovering explicit mathematical expressions directly from the input data, therefore enabling better model interpretability while maintaining predictive performance.

This thesis provides a systematic study of Symbolic Regression methods for surrogate modeling. The main goal is to compare open-source SR frameworks that employ different algorithmic approaches, including evolutionary, genetic programming, transformation-based, and neural methods. The evaluated frameworks are gplearn, Operon, PySR, GeneticEngine, ITEA, and EQL.

To evaluate these frameworks, a benchmark is developed using datasets from different domains, including engineering, physics, material science and energy systems. The selected SR models are evaluated using identical training and test data and a common evaluation protocol with documented algorithm-specific search budgets. The evaluation considers predictive accuracy, computational efficiency, model complexity, stability across repeated trials, and sensitivity to input scaling. The thesis investigates the trade-offs between accuracy and interpretability, as well as the current challenges and research gaps in Symbolic Regression for surrogate modeling.

TODO: Add Results and Conclusion to Abstract.

# Contents

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Abstract</b>                                           | <b>II</b> |
| <b>List of Figures</b>                                    | <b>V</b>  |
| <b>List of Tables</b>                                     | <b>VI</b> |
| <b>1 Introduction</b>                                     | <b>1</b>  |
| 1.1 Motivation . . . . .                                  | 1         |
| 1.2 The Need for Interpretable Models . . . . .           | 2         |
| 1.3 Symbolic Regression as a Candidate Approach . . . . . | 2         |
| 1.4 Research Question and Contributions . . . . .         | 3         |
| <b>2 Background</b>                                       | <b>5</b>  |
| 2.1 Surrogate Modeling . . . . .                          | 5         |
| 2.2 Symbolic Regression . . . . .                         | 6         |
| 2.3 Main Symbolic Regression Method Families . . . . .    | 7         |
| 2.4 The Accuracy Complexity Trade-Off . . . . .           | 8         |
| 2.5 Interpretability in Scientific Modeling . . . . .     | 9         |
| <b>3 Related Work</b>                                     | <b>11</b> |
| 3.1 Symbolic Regression Reviews . . . . .                 | 11        |
| 3.2 Benchmark Studies . . . . .                           | 12        |
| 3.3 Applications in Science and Engineering . . . . .     | 13        |
| 3.4 Selected Framework Publications . . . . .             | 14        |
| <b>4 Methodology</b>                                      | <b>17</b> |
| 4.1 Framework Selection Criteria . . . . .                | 17        |
| 4.2 Dataset Selection Criteria . . . . .                  | 17        |
| 4.3 Benchmark Protocol . . . . .                          | 18        |
| 4.4 Evaluation Metrics . . . . .                          | 20        |
| 4.5 Reproducibility Setup . . . . .                       | 21        |
| 4.6 Interpretability Rubric . . . . .                     | 21        |
| <b>5 Experiments</b>                                      | <b>23</b> |
| 5.1 Experimental Matrix and Task Roles . . . . .          | 23        |
| 5.2 Framework Configurations . . . . .                    | 24        |
| 5.3 Execution Procedure and Environments . . . . .        | 25        |

## Contents

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| 5.4      | Repeated Runs and Matrix Coverage . . . . .                 | 26        |
| 5.5      | Target-Noise Experiments . . . . .                          | 27        |
| 5.6      | Logistic-Map Rollout Experiment . . . . .                   | 27        |
| 5.7      | Diagnostic and Excluded Experiments . . . . .               | 28        |
| <b>6</b> | <b>Results</b>                                              | <b>29</b> |
| 6.1      | Known-Equation Recovery . . . . .                           | 29        |
| 6.2      | Predictive Performance on Empirical Tasks . . . . .         | 30        |
| 6.3      | Computational Efficiency and Practical Execution . . . . .  | 31        |
| 6.4      | Equation Complexity and Interpretability Evidence . . . . . | 32        |
| 6.5      | Noise Robustness and Seed Stability . . . . .               | 33        |
| <b>7</b> | <b>Discussion</b>                                           | <b>35</b> |
| 7.1      | Interpretation of Framework Trade-Offs . . . . .            | 35        |
| 7.2      | Suitability for Surrogate-Modelling Tasks . . . . .         | 35        |
| 7.3      | Practical Framework Selection . . . . .                     | 35        |
| 7.4      | Limitations and Threats to Validity . . . . .               | 35        |
| 7.5      | Research Gaps and Future Benchmarking Needs . . . . .       | 35        |
| <b>8</b> | <b>Conclusion</b>                                           | <b>36</b> |
| 8.1      | Summary of Findings . . . . .                               | 36        |
| 8.2      | Answer to the Research Question . . . . .                   | 36        |
| 8.3      | Future Work . . . . .                                       | 36        |
|          | <b>Bibliography</b>                                         | <b>37</b> |

# List of Figures

# List of Tables

|     |                                                                                                                                                                                                 |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 5.1 | Main framework configurations used in the experiments. . . . .                                                                                                                                  | 24 |
| 5.2 | Completed run coverage in the primary experiment matrix. . . . .                                                                                                                                | 26 |
| 6.1 | Clean logistic-map results. Repeated values are means; $n$ is the number of completed seeds. . . . .                                                                                            | 30 |
| 6.2 | Clean-test $R^2$ on empirical datasets. Values for DSR and NeSymRe, and all values on the two energy replacement tasks, are single seed-42 runs; the other values are three-seed means. . . . . | 30 |



# 1 Introduction

## 1.1 Motivation

Scientific and engineering research increasingly relies on numerical simulations and physical experiments to investigate complex systems, optimize technical designs, and evaluate control strategies. High-fidelity simulations may require expensive numerical solvers, while physical experiments may be time-consuming, costly, or destructive. These costs become a practical bottleneck when a system must be evaluated repeatedly, for example during optimization, uncertainty quantification, sensitivity analysis, or parameter exploration [2].

Surrogate modeling addresses this bottleneck by learning an inexpensive approximation of the input output relationship represented by a costly simulation or experimental process. A surrogate is trained on observations obtained from simulations, physical measurements, or both, and is then used to provide inexpensive predictions for new inputs. This approach is widely used to manage computational complexity in engineering design and simulation-based analysis, where the cost of generating training data must be balanced against predictive accuracy and the computational savings achieved during repeated evaluation [2]. Surrogate models are particularly useful in domains such as physics, mechanical engineering, materials science and energy systems, where obtaining direct simulation results or experimental measurements can be costly.

Modern machine learning methods are common choices for surrogate modeling because they can approximate complex nonlinear relationships. Neural networks, ensemble models, Gaussian processes, and other flexible approaches achieve strong predictive performance. However, this flexibility can make the resulting surrogate difficult to inspect, explain, or compare with known scientific relationships. This lack of transparency may be acceptable when a surrogate is used only for prediction. However, when the model is intended to support scientific understanding, engineering decisions, or safety-relevant applications, predictive accuracy alone may be insufficient, and the structure and behavior of the learned model must also be considered.

## 1.2 The Need for Interpretable Models

Interpretability becomes important when a model is intended to support scientific understanding rather than prediction alone. In scientific and engineering applications, an interpretable surrogate can help researchers inspect which variables appear in the learned relationship, examine their nonlinear interactions, and compare the model structure with established domain knowledge. These forms of inspection are difficult when the surrogate is a black box. A low prediction error on testing data does not by itself demonstrate that a model has learned a scientifically meaningful or domain-consistent relationship.

The distinction between interpretable models and post-hoc explanations is important. Rudin argues that in high-stakes settings, one should prefer models that are interpretable by design instead of first training a black-box model and then trying to explain it afterward [17]. This argument is also relevant for scientific surrogate modeling. If the learned model is meant to support reasoning about a physical or engineering system, the transparency of the model itself is important.

Interpretability is not a single property. A model may be considered interpretable because it is simple, because it uses familiar mathematical operations, because its variables have clear roles or because the complete model can be written down and evaluated by a human. In surrogate modeling, interpretability must also be balanced against accuracy. A very simple model may be easy to understand but too inaccurate to be useful, while a highly accurate model may be too complex to provide valuable insight. This trade-off motivates the benchmarking of methods according to both predictive accuracy and model complexity [13, 9].

## 1.3 Symbolic Regression as a Candidate Approach

Symbolic Regression (SR) is a machine learning approach that searches for explicit mathematical expressions fitting a given dataset. Instead of assuming a fixed model form, such as a linear model or a predefined polynomial basis, SR explores combinations of variables, constants, and mathematical operators. The result is not only a predictor but also an equation that can be read, evaluated, simplified, and compared with domain knowledge [14, 10].

This makes SR a promising candidate for interpretable surrogate modeling. A symbolic expression can often be much cheaper to evaluate than the original simulation and more transparent than the black-box models. Because SR produces explicit mathematical expressions, the influence of individual variables and their nonlinear interactions can be inspected directly. If the resulting expression is sufficiently simple, it can also be compared with known scientific relationships and domain knowledge. SR has therefore received considerable attention in scientific discovery,

## 1 Introduction

including physics, materials science, and dynamical systems [3, 12, 21, 4].

Several well-known examples illustrate the scientific motivation for SR. AI Feynman uses physics-inspired strategies to recover equations from the Feynman Lectures on Physics, showing how prior structural ideas such as symmetry and separability can support symbolic equation discovery [20]. Sparse Identification of Nonlinear Dynamics (SINDy) similarly pursues interpretable governing equations from data, although it uses sparse regression over a predefined library of candidate terms rather than general symbolic search [5]. These works show the broader appeal of data-driven equation discovery for scientific modeling.

At the same time, SR is not automatically superior to other surrogate modeling approaches. The search space of possible expressions is large, and different frameworks use different strategies to explore it. Evolutionary algorithms, genetic programming variants, and neural or reinforcement-learning-based approaches differ in their assumptions, computational cost, ease of use, and ability to discover compact expressions [10, 16]. The quality of the final model also depends on operator choices, complexity penalties, training budgets, and implementation details. Therefore, a systematic comparison is important, before SR can be recommended as a practical surrogate modeling tool.

### 1.4 Research Question and Contributions

The central research question of this thesis is:

*How suitable are Symbolic Regression frameworks for interpretable surrogate modeling, and how do they compare with respect to predictive performance, computational cost, equation complexity, and practical usability?*

**TODO: Add citations of algorithm papers.**

To answer this question, this thesis conducts a systematic study of six open-source SR frameworks: gplearn, Operon, PySR, GeneticEngine, ITEA, and EQL. The selected frameworks represent different approaches to symbolic model discovery, including tree-based genetic programming, efficient evolutionary search, grammar-guided genetic programming, interaction-transformation-based evolutionary search, and neural equation learning. PySR provides a scientific SR interface built around SymbolicRegression.jl [7], while Operon implements a computationally efficient genetic programming approach in C++ [6]. The frameworks are evaluated on eight analytical, simulated, and empirical surrogate-modeling problems using fixed training and test datasets and documented algorithm-specific search budgets. The comparison considers predictive accuracy, computational efficiency, expression complexity, recovery of known equations, stability across repeated trials, sensitivity to input scaling, and practical reproducibility.

## 1 Introduction

The main contributions of this thesis are:

- a structured overview of Symbolic Regression as an approach for interpretable surrogate modeling;
- a benchmark setup for comparing selected SR frameworks on datasets from different application domains;
- an evaluation protocol covering predictive accuracy, runtime, setup effort, equation complexity, and stability in repeated prediction settings;
- an interpretability-oriented comparison of the discovered symbolic expressions;
- practical guidance on strengths, limitations, and challenges of the current SR frameworks for surrogate modeling.

The remainder of this thesis is organized as follows. Chapter 2 introduces the background on surrogate modeling, Symbolic Regression, method families, and interpretability. Chapter 3 discusses related work, including existing reviews, benchmark studies, and applications. Chapter 4 describes the methodology, including framework and dataset selection, evaluation metrics, reproducibility setup, and interpretability. Chapter 5 presents the experimental setup. Chapter 6 reports the results. Chapter 7 discusses the findings and their limitations. Chapter 8 concludes the thesis and outlines directions for future work.

## 2 Background

### 2.1 Surrogate Modeling

Many scientific and engineering workflows rely on simulation models that provide detailed representations of physical systems but are computationally expensive to evaluate. Finite element analysis and computational fluid dynamics are prominent examples. Their computational cost becomes a practical limitation when numerous evaluations are required for tasks such as design optimization, uncertainty quantification, sensitivity analysis, or exploration of a parameter space [2].

A surrogate model approximates the input output relationship of a computationally expensive simulation or physical process. It is constructed from a finite set of observations and is intended to provide predictions at a substantially lower evaluation cost than the original process [2]. If the original model is denoted by

$$y = f(\mathbf{x}), \tag{2.1}$$

where  $\mathbf{x} \in \mathbb{R}^d$  is an input vector and  $y \in \mathbb{R}$  is the corresponding output, a surrogate model approximates this relationship with a function  $\hat{f}$  such that

$$\hat{y} = \hat{f}(\mathbf{x}) \approx f(\mathbf{x}). \tag{2.2}$$

The main practical advantage of a surrogate is that evaluating  $\hat{f}$  is generally less computationally expensive than evaluating the original model  $f$ . This makes repeated model evaluations feasible, but computational efficiency alone does not determine whether a surrogate is useful. Its predictive accuracy on relevant unseen data, the amount of data required for its construction, and the computational cost of fitting and evaluating it must also be considered [2].

A wide range of statistical and machine-learning methods can be employed as surrogate models, including polynomial response surfaces, Gaussian processes, artificial neural networks, support vector regression, boosted trees, and random forests [2]. These methods can approximate complex nonlinear relationships without requiring a complete analytical description of the underlying process. However, flexible predictive models may provide limited insight into the learned input output relationship, even when they achieve low prediction error [14].

## 2 Background

This thesis focuses on evaluating Symbolic Regression as an approach to surrogate modeling, with interpretability considered alongside predictive performance and computational efficiency. In addition to prediction error, the evaluation therefore examines whether the resulting mathematical expressions can be inspected, simplified, and compared with available domain knowledge. Symbolic Regression is relevant in this context because it searches for explicit mathematical expressions directly from data, providing a model representation that may support such analysis [14].

### 2.2 Symbolic Regression

Symbolic Regression (SR) is a supervised learning approach that searches for mathematical expressions fitting a given dataset. In contrast to ordinary regression methods, SR does not assume a fixed functional form in advance. For example, a linear regression model assumes that the target can be represented as a weighted sum of input variables. A polynomial regression model assumes a predefined set of polynomial terms. SR instead searches over both the structure of the expression and its numerical parameters [3, 13].

Symbolic regression (SR) searches for a mathematical expression that describes the relationship between input variables and a target variable directly from data [14]. Consider a supervised dataset

$$D = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad (2.3)$$

where  $D$  denotes the complete dataset,  $N$  is the number of observations, and  $i \in \{1, \dots, N\}$  indexes an individual observation. The vector  $\mathbf{x}_i \in \mathbb{R}^d$  contains the values of the  $d$  input variables for the  $i$ -th observation, while  $y_i \in \mathbb{R}$  is its corresponding target value.

The objective of SR is to find both a symbolic expression structure  $\phi$  and numerical constants  $\boldsymbol{\theta}$  such that the prediction  $\phi(\mathbf{x}_i; \boldsymbol{\theta})$  approximates  $y_i$ . For example, one possible measure of prediction error is the mean squared error (MSE),

$$\mathcal{L}_{\text{MSE}}(\phi, \boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N (y_i - \phi(\mathbf{x}_i; \boldsymbol{\theta}))^2. \quad (2.4)$$

where  $y_i - \phi(\mathbf{x}_i; \boldsymbol{\theta})$  denotes the prediction error for the  $i$ -th observation.

However, minimizing prediction error alone can produce unnecessarily large expressions. Because the result of SR is intended to remain interpretable, expression complexity must also be considered. Complexity may, for example, be measured

## 2 Background

by the number of nodes or operators in the expression tree. SR is therefore commonly treated as a multi objective problem involving a trade-off between predictive accuracy and expression complexity [7, 13].

Candidate expressions are commonly represented as syntax trees. In such a tree, internal nodes are mathematical operators such as  $+$ ,  $-$ ,  $\times$ , division,  $\sin$ ,  $\cos$ ,  $\log$ , or  $\exp$ , while leaf nodes are variables or constants. For example, the expression

$$\phi(x_1, x_2) = 3x_1 + \sin(x_2/5) \quad (2.5)$$

can be represented as a tree combining multiplication, division, addition, two constants, two variables, and a sine operator.

**TODO: Add an actual tree diagram.**

This representation makes it possible to modify expressions by changing subtrees, replacing operators, or adjusting constants.

The search space of SR is very large. Even a small set of variables and operators can generate many possible expressions, and the number of possible expressions grows quickly with expression size. This is one reason why SR is computationally difficult and why many frameworks use different search strategies [10]. Another challenge is the treatment of constants. In scientific equations, real-valued constants are often essential, but searching simultaneously over expression structure and continuous constants is much harder than fitting coefficients in a fixed model form [7].

Despite these challenges, SR is useful for scientific modeling because its outputs can be interpreted directly. A discovered equation can be evaluated cheaply, simplified symbolically, compared with existing theory, inspected for asymptotic behavior, and checked for physically implausible terms. This distinguishes SR from many black-box methods, where interpretability is usually added after training through explanation methods rather than being part of the model itself [14].

### 2.3 Main Symbolic Regression Method Families

Recent surveys group SR methods into several broad families. The most relevant distinction for this thesis is between deterministic methods, genetic programming methods, and neural methods [10]. These families differ in how they explore the expression space and handle constants.

Deterministic methods use systematic search, enumeration, sparse regression, mixed-integer optimization, or related strategies to explore candidate expressions. Their main advantage is that they can be more stable and reproducible than purely stochastic methods, because the search process is less dependent on random initialization. Some deterministic approaches can also provide stronger guarantees

## 2 Background

within a restricted search space. The limitation is that the number of possible expressions grows rapidly with the number of variables, operators, and expression depth. As a result, deterministic methods may become computationally expensive for high-dimensional or complex problems [10].

Genetic Programming (GP) methods are historically the most established family of SR algorithms. GP treats expressions as individuals in a population and evolves them over many generations. Candidate expressions are evaluated according to a fitness function, selected, recombined, mutated, and simplified. This family is flexible because it can search over many different expression structures without requiring a fixed basis. It also naturally supports multi-objective optimization, where accuracy and complexity are optimized together [3, 13].

**TODO: Maybe swap DSR example for EQL.**

Neural-network-based SR methods use neural networks to guide or perform the expression search. Instead of evolving expressions only through classical GP operators, these methods may train a recurrent neural network, transformer, or other neural architecture to generate candidate formulas. Deep Symbolic Regression (DSR), for example, uses reinforcement learning and a generative recurrent neural network to search for expressions [14, 13]. Neural approaches are motivated by the ability of deep learning systems to learn search heuristics from data or from many generated examples. Their limitations include training complexity, sensitivity to the distribution of training problems, and the risk that strong performance on synthetic benchmarks does not directly transfer to scientific data.

The variety of method families is important for this thesis because there is no single best SR approach. A framework that performs well on one type of problem may behave differently on datasets with noisy data or high-dimensional data. Therefore, the comparison in this thesis treats SR frameworks not only as implementations of one idea, but as tools with different algorithmic assumptions.

### 2.4 The Accuracy Complexity Trade-Off

The central evaluation problem in SR is the trade-off between predictive accuracy and expression complexity. If only accuracy is optimized, SR can produce long expressions that fit the training data well but are difficult to interpret and prone to overfitting. If only simplicity is optimized, the resulting expression may be readable but too inaccurate to serve as a useful surrogate. A meaningful SR model should therefore achieve an appropriate balance between these objectives.

Complexity can be measured in different ways. A common syntactic measure is the number of nodes, operators, variables, and constants in an expression. Other measures consider expression depth, operator types, semantic behavior, nonlinearity, or the effective difficulty of understanding the formula [13, 10]. There is no perfect



## 2 Background

complexity metric. A short equation can still be difficult to interpret if it contains unfamiliar or unstable operations, while a slightly longer equation may be more meaningful if it matches known domain structure.

Many SR frameworks return not only one expression but a set of candidate expressions along a Pareto front. A Pareto front contains models for which no other model is both more accurate and less complex. This is useful because the final choice of model depends on the application. For a deployment-focused surrogate, a slightly more complex expression may be acceptable if it improves predictions significantly. For scientific insight, a simpler expression may be preferable if it captures the dominant relation and remains physically plausible.

In this thesis, the accuracy complexity trade-off is treated as a core evaluation dimension. Predictive metrics such as mean squared error or coefficient of determination describe how well a model fits the data, while complexity metrics describe how complex the resulting expression is. Considering both dimensions is essential because a symbolic surrogate is only useful if it is accurate enough to approximate the underlying process and simple enough to support interpretation.

### 2.5 Interpretability in Scientific Modeling

Interpretability is a major motivation for applying SR to surrogate modeling. In scientific contexts, models are often expected to do more than predict. They should help explain a underlying system, reveal relationships between variables, and support reasoning about behavior in new regimes. A symbolic expression is attractive because the full mapping from inputs to outputs is visible in mathematical form.

The distinction between interpretability and post-hoc explainability is important. A black-box model may be explained after training using feature importance scores, saliency methods, local approximations, or other explanation techniques. These methods can be useful, but they do not make the original model transparent. An interpretable model, by contrast, exposes its structure directly. SR belongs to this second category, as it can produce sufficiently compact and meaningful expressions [14].

SR can support scientific discovery when the resulting expression reveals a relation that was not explicitly provided to the model. Reviews of SR in physical sciences emphasize this potential. SR can generate analytical equations from data and thereby connect data-driven modeling with the traditional scientific practice of formulating laws and empirical relations [3, 14]. At the same time, interpretability should not be assumed automatically. A symbolic expression may be mathematically explicit but still too complex, unstable, or domain-inconsistent to be useful.

This thesis treats interpretability as an evaluation criterion rather than as a guaranteed outcome. The discovered expressions will be compared in terms of their size,

## *2 Background*

structure, operator usage, and practical readability. This allows SR frameworks to be evaluated not only by whether they predict accurately, but also by whether they produce surrogate models that can plausibly support scientific understanding.

## 3 Related Work

### 3.1 Symbolic Regression Reviews

The recent growth of Symbolic Regression (SR) has led to several survey papers that organize the field and summarize its main technical directions. Makke and Chawla provide a broad review of SR as a method for interpretable scientific discovery, emphasizing the ability of SR to infer compact mathematical expressions directly from data [14]. Their review discusses the historical development of SR, its relation to genetic programming, the emergence of deep-learning-based approaches, and applications in scientific domains. This perspective is closely aligned with the motivation of this thesis, because surrogate modeling in science and engineering often requires models that are not only accurate but also interpretable.

Angelis et al. focuses specifically on SR in the physical sciences [3]. Their review positions SR as a response to the limitations of black-box machine learning models in physics-based computation. They discuss SR as a data-driven method that can generate analytical equations and thereby support interpretation, generalization, and the discovery or verification of physical relations.

Dong and Zhong provide a more recent and comprehensive survey of advances in SR [10]. They organize SR methods into deterministic, genetic-programming-based, and neural-network-based families, and they also summarize benchmark datasets, evaluation metrics, software tools, and open research challenges. Their survey is useful for understanding the current methodological landscape and for motivating the framework selection in this thesis. In particular, the distinction between GP-based and neural SR approaches is relevant because the evaluated frameworks represent different search and implementation strategies.

These reviews show that SR is no longer limited to genetic programming. Current SR research includes evolutionary algorithms, deterministic search, sparse regression, reinforcement learning, transformer-based approaches, and hybrid methods. At the same time, the reviews also point to persistent challenges. For example, high computational cost, sensitivity to benchmark design, difficulty in evaluating interpretability, and inconsistent behavior across datasets.

## 3.2 Benchmark Studies

Benchmarking is a central issue in SR research because methods are often evaluated on different datasets, with different computational budgets, and with different definitions of success. Early SR papers frequently used small synthetic benchmarks, which made results difficult to compare across studies. Orzechowski, La Cava, and Moore addressed this issue by evaluating recent SR methods against standard machine learning regressors on a large set of regression problems [15]. Their study compared four GP-based SR methods with several scikit-learn regression methods on 94 real-world datasets. The results showed that SR methods can be competitive with strong machine learning baselines, but also that runtime remains as a limitation.

La Cava et al. extended this line of work with SRBench, a larger and more systematic benchmark for symbolic regression [13]. SRBench compares 14 SR methods and 7 machine learning methods across 252 regression problems, including real-world datasets, ground-truth symbolic equations, physics equations, and systems of ordinary differential equations. Importantly, SRBench evaluates not only predictive accuracy but also expression complexity and symbolic solution recovery. The benchmark therefore provides a basis for comparing SR methods in terms of both predictive performance and expression complexity.

The SRBench results also highlight that no single method is universally optimal. Some frameworks perform strongly on black-box regression tasks, while others are better at recovering exact equations under controlled conditions. Operon, for example, is reported as one of the strongest methods on real-world black-box regression problems and as part of the best accuracy-simplicity trade-offs [13]. These findings motivate a more focused comparison of selected frameworks in the surrogate modeling context, where practical usability, and interpretability are important.

More recent work has extended and criticized existing benchmark practices. SRBench++ argues that expression size alone is an insufficient proxy for interpretability and introduces domain-expert interpretation into the benchmarking process [9]. It also evaluates SR algorithms on more specific task properties, such as feature selection, robustness, extrapolation, and local minima. This is important because a symbolic expression can be short but still scientifically unhelpful, or slightly longer but more meaningful to a domain expert.

Aldeia et al. present a call for the next generation of SR benchmarks, arguing that SRBench should evolve as a living benchmark [1]. Their work expands the set of evaluated methods, refines evaluation metrics, and considers trade-offs between accuracy, complexity, and energy consumption. The study emphasizes standardization of hyperparameter tuning, execution constraints, computational resources, and benchmark maintenance. These issues are directly relevant for this thesis because a fair comparison of SR frameworks requires a reproducible protocol.

### 3 Related Work

Overall, benchmark studies show that SR evaluation is more complex than ordinary regression benchmarking. Accuracy, model complexity, solution recovery, runtime, robustness, and interpretability all matter. The present thesis builds on these insights but uses a narrower experimental scope. Instead of ranking many SR methods across hundreds of datasets, it compares a selected set of open-source frameworks under a reproducible surrogate modeling protocol.

## 3.3 Applications in Science and Engineering

TODO: Maybe think about removing this section since it is not directly connected to what is done in this thesis.

SR has a long connection to scientific discovery because many scientific explanations are expressed as mathematical equations. Reviews of SR applications show examples in physics, chemistry, materials science, engineering, medicine, climate science, and other domains [14, 3]. In these settings, the value of SR is not limited to prediction. A discovered expression may reveal variable interactions, suggest scaling relations, support extrapolation, or provide a compact surrogate for a costly simulation.

One closely related research direction is the discovery of governing equations from data. Brunton, Proctor, and Kutz introduced Sparse Identification of Nonlinear Dynamics (SINDy), which uses sparse regression to identify parsimonious dynamical systems from data [5]. SINDy differs from general SR because it assumes a library of candidate functions and searches for a sparse combination of terms. Nevertheless, it shares the same scientific goal: obtaining interpretable equations that balance accuracy and model complexity. For this thesis, SINDy is relevant as a neighboring approach to symbolic surrogate modeling and equation discovery.

The application literature also demonstrates why interpretability must be treated carefully. In scientific modeling, a model is not useful merely because it is explicit. It should also be consistent with domain knowledge, behave reasonably outside the training data, and avoid unnecessary complexity. This concern appears repeatedly in SR reviews and benchmark papers [14, 9]. It is especially important for surrogate modeling, where the learned model may be used for optimization or repeated prediction in regions not densely covered by training data.

PySR was developed explicitly with scientific equation discovery in mind [7]. Its design emphasizes practical requirements that often arise in scientific applications, such as real-valued constants, custom operators, custom losses, and the ability to inspect a Pareto front of candidate equations. This practical focus makes PySR particularly relevant for surrogate modeling, where the final expression should be both predictive and usable by researchers.

The broader scientific application literature therefore supports the central premise of this thesis: SR is promising for surrogate modeling because it can produce compact

analytical approximations, but its usefulness depends on the quality of the discovered expressions, the stability of the framework, and the trade-off between accuracy and interpretability.

## 3.4 Selected Framework Publications

This thesis evaluates six open-source Symbolic Regression frameworks: `gplearn`, `Operon`, `PySR`, `GeneticEngine`, `ITEA`, and `EQL`. They were selected to represent different approaches to symbolic model construction, including conventional tree-based genetic programming, performance-oriented evolutionary search, grammar-guided genetic programming, interaction transformation expressions, and neural equation learning. The associated publications describe the algorithms and software architectures, while the exact versions and configurations used in this thesis are specified separately in the experimental methodology.

### **`gplearn`**

`Gplearn` is a Python implementation of conventional tree-based genetic programming with an interface modeled after `scikit-learn` [19]. Candidate expressions are represented as programs and evolved using tournament selection, crossover, and several forms of mutation, while a parsimony coefficient can be used to discourage unnecessarily large expressions. Unlike the other selected frameworks, `gplearn` does not have a dedicated peer-reviewed framework publication. However, it is included and characterized as a Koza-style symbolic-regression implementation in the `SRBench` comparison [13]. Its comparatively direct implementation and familiar estimator interface make it a useful reference point for evaluating more specialized SR approaches.

### **`Operon`**

`Operon` is an efficient C++ genetic programming framework designed specifically for Symbolic Regression [6]. It represents expression trees as contiguous linear sequences, which improves memory locality and enables efficient evaluation and manipulation of candidate models. Its modular evolutionary design is organized around offspring generators and supports automatic differentiation and gradient-based optimization of numerical constants. `Operon` emphasizes runtime performance, memory efficiency, and extensibility, making it relevant for examining how an optimized genetic programming implementation performs within the benchmark.

### 3 Related Work

#### **PySR**

PySR is an open-source library for scientific Symbolic Regression that provides a Python interface to the Julia backend SymbolicRegression.jl [7]. Its search procedure uses multiple evolving populations and an evolve simplify optimize loop in which candidate expressions are evolved, simplified, and refined through numerical constant optimization. PySR also supports user defined operators, losses, complexity measures, and expression export to several scientific-computing formats. These capabilities make it a flexible framework to search for symbolic models under application specific mathematical constraints.

#### **GeneticEngine**

GeneticEngine is a Python framework for grammar-guided genetic programming in which the search space can be defined using Python data types rather than a separate textual grammar [11]. The framework combines ideas from grammar-guided and strongly typed genetic programming, allowing the structures and constraints of valid candidate programs to be expressed through the host language’s type system. The publication emphasizes the ergonomics and expressive power of this representation and demonstrates this through an open source Python implementation.

#### **ITEA**

The Interaction Transformation Evolutionary Algorithm (ITEA) searches within a restricted representation in which models are formed as linear combinations of transformed interactions between input variables [8]. Each interaction is defined through powers of the input variables and a selected transformation function, while the corresponding linear coefficients are fitted numerically. Rather than using conventional subtree crossover, ITEA evolves a population through mutation operations that add, remove, or modify terms, interactions, and transformations. This structured representation distinguishes ITEA from general expression-tree methods and provides direct control over the maximum number of terms in a model.

#### **EQL**

Equation Learning (EQL) represents mathematical expressions through a differentiable neural network whose units implement elementary operations such as identity, trigonometric functions, and multiplication [18]. Alternating linear transformations and operation-specific units allow the network to construct nonlinear analytical relationships, while sparsity regularization encourages a small number of active connections and therefore more concise expressions. The architecture is optimized using gradient-based methods rather than an evolutionary population, making EQL

### 3 *Related Work*

methodologically distinct from the other frameworks in the benchmark. The evaluated adapter implements the Equation Learner approach described in the associated literature.

The selected frameworks therefore cover several parts of the contemporary SR landscape. `gplearn`, `Operon`, `PySR`, and `GeneticEngine` use different forms of genetic or evolutionary search, `ITEA` evolves expressions within the more restricted interaction transformation representation and `EQL` constructs equations through a differentiable neural architecture.



## 4 Methodology

### 4.1 Framework Selection Criteria

The purpose of the benchmark is not to identify a universally best Symbolic Regression (SR) algorithm, but to compare representative frameworks under conditions relevant to surrogate modeling. Framework selection was guided by five criteria: (i) the method is described in a scientific publication or authoritative software documentation, (ii) an open-source implementation is publicly available, (iii) the implementation returns an explicit mathematical expression rather than predictions alone, (iv) it can be integrated into the common SRBench-derived evaluation pipeline, and (v) it contributes a relevant algorithmic or representational approach to the comparison. Practical installability, compatibility with the evaluation interface, and reproducible execution were also considered because technical failures and unavailable model expressions affect the usefulness of a framework in practice. This reflects recommendations that SR benchmarks should consider reproducibility, complexity, and practical execution in addition to predictive accuracy [9, 1].

Based on these criteria, six frameworks were included: gplearn, Operon, PySR, GeneticEngine, ITEA, and EQL. Together, they provide a range of evolutionary, representation-based, and gradient-based approaches to Symbolic Regression. Their exact configurations and execution environments are described in Chapter 5.

### 4.2 Dataset Selection Criteria

The benchmark problems were selected to represent scientific and engineering surrogate-modeling tasks while remaining feasible for repeated Symbolic Regression searches. Each problem was required to have numerical input variables, a continuous target, clearly documented feature meanings and domains, and a reproducible data source or generating function. The available data also had to support disjoint training and test sets without categorical encoding, missing-value imputation, or extensive feature engineering. Documented input domains were additionally required so that the effect of fixed domain-based input scaling could be evaluated consistently.

The selection combines problems with known analytical equations and problems for which no privileged ground-truth expression is available. Cantilever, Borehole, Piston, and Wing Weight provide analytical generating equations and therefore support

## 4 Methodology

both predictive evaluation and symbolic recovery analysis. Combined Cycle Power Plant, Naval Propulsion, Gas Turbine NOx Emissions, and Concrete Compressive Strength represent simulator-derived, measured, or experimental relationships for which evaluation is based on held-out predictive performance, expression complexity, runtime, and stability rather than exact equation recovery.

The final set of eight problems covers several engineering domains, between three and ten input variables, different physical scales, and mathematical relationships of varying structural difficulty. The analytical problems include multiplication, division, integer and fractional powers, logarithms, square roots, and trigonometric terms, while the non-analytical problems introduce measurement variation and relationships not defined by a known closed-form equation. This combination enables the frameworks to be assessed both as equation-recovery methods and as practical data-driven surrogate models. The exact problem definitions, sample sizes, data sources, and train test construction procedures are presented in Chapter 5.

### 4.3 Benchmark Protocol

All experiments use a common SRBench derived evaluation pipeline. Each problem has a fixed training table and a separate test table that are generated or selected before the algorithm repetitions are executed. Dataset construction uses documented problem specific seeds, whereas the repetition seed controls the symbolic regression algorithm where supported. Consequently, every framework and repetition receives identical observations, preventing variation in dataset sampling from being confused with variation in algorithm behavior. SHA-256 hashes of the uncompressed training and test tables are stored in each result to make this invariant auditable.

Every algorithm–problem combination is evaluated under two input-scaling conditions. In the **raw** condition, the stored inputs are supplied to the estimator in their original physical units. In the **domain\_minmax** condition, each input  $x_j$  with documented lower and upper bounds  $l_j$  and  $u_j$  is mapped to  $[-1, 1]$  according to

$$z_j = 2 \frac{x_j - l_j}{u_j - l_j} - 1. \quad (4.1)$$

The transformation uses fixed problem-domain bounds rather than statistics estimated from the training data, ensuring that it is identical across frameworks and repetitions and does not introduce test-data leakage. Target values are not scaled and remain in their original units under both conditions. Expressions learned from normalized inputs are transformed back to the original physical variables during symbolic post-processing.

The final repetition protocol uses the predefined labels 2, 3, 5, 7, 11, 13, 17, 19, 23, and 29. These labels are passed to each estimator that exposes a supported random-

## 4 Methodology

seed parameter. The ITEA implementation does not expose such a parameter, so its ten executions are reported as uncontrolled repetitions rather than seed-controlled trials. With six algorithms, eight problems, two input-scaling conditions, and ten repetitions, the complete benchmark contains 960 expected trials.

Each trial fits one model using the fixed training target and evaluates it exclusively on the corresponding held-out reference-test set. A successful result, an explicit execution failure, and an absent trial are treated as three separate states. Failure records retain the algorithm, problem, scaling condition, repetition label, and process exit status, while the summary tables report successful, failed, and missing trials separately. Failed or missing runs are therefore not silently excluded from the reported experiment coverage.

Each framework uses one explicitly documented configuration across the benchmark, and no hyperparameter optimization is performed using the final test results. The configured search effort is expressed through algorithm-specific controls, such as population sizes, generations, evaluation limits, iterations, or time limits. These settings were selected to make repeated execution feasible and do not impose equal computational budgets across frameworks. Runtime comparisons therefore describe the evaluated configurations and must not be interpreted as either equal-compute comparisons or estimates of each framework’s best attainable performance.

Shared execution controls restrict benchmark containers to one thread and request five complete predictions of the reference-test set. Model fitting, prediction, and expression extraction are timed within the running algorithm environment. Container startup, image retrieval, dataset generation, result serialization, and symbolic post-processing are excluded from the reported algorithm runtime.

Symbolic parsing, transformation to physical coordinates, simplification, complexity measurement, and comparison with available ground-truth equations are performed as separate post-processing steps. Each stage has an independent timeout and writes a versioned analysis record without modifying the original trial result. Separating expression analysis from model execution ensures that a parsing or simplification failure does not invalidate an otherwise successful training and prediction trial.

## 4.4 Evaluation Metrics

Predictive performance is calculated exclusively on the held-out, noise-free test set. For observations  $y_i$ , predictions  $\hat{y}_i$ , and test-set mean  $\bar{y}$ , the benchmark reports

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (4.2)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (4.3)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}. \quad (4.4)$$

RMSE emphasises large errors, MAE gives a more robust measure in the target’s original unit, and  $R^2$  allows a dimensionless comparison of fit quality within each dataset. Because target scales differ, raw RMSE and MAE are not ranked across datasets.

Computational effort is measured with a monotonic wall clock. Training time covers the adapter’s model-fitting call. Prediction time covers prediction of the complete test set and is additionally divided by the number of test samples. Data download, parsing, and metric computation are excluded. Memory consumption was considered during benchmark design but is not collected by the implemented pipeline and is therefore not used in the comparison.

Every returned equation is parsed with SymPy to obtain framework-independent structural measures. Equation size is the total number of nodes in the SymPy expression tree, depth is the longest root-to-leaf path, and the used operator set is collected from the parsed and textual expression. A parse failure is stored explicitly instead of assigning a misleading complexity value. These measures provide approximations of syntactic complexity; they do not by themselves establish whether an equation is scientifically meaningful.

Noise robustness is summarised by the absolute RMSE increase, the ratio of noisy-run to clean-run RMSE, and the decrease in  $R^2$ :

$$\Delta \text{RMSE} = \text{RMSE}_{\text{noisy}} - \text{RMSE}_{\text{clean}}, \quad (4.5)$$

$$\Delta R^2 = R^2_{\text{clean}} - R^2_{\text{noisy}}. \quad (4.6)$$

For repeated experiments, the analysis reports mean, sample standard deviation, minimum, and maximum for predictive, timing, complexity, robustness, and rollout measures.

For logistic-map rollouts, RMSE and MAE are retained at every time step. Three summary quantities are reported: mean rollout RMSE over steps 1–50, RMSE at the final step, and the maximum absolute error over all cases and rollout steps. These

metrics assess accumulated simulation error and complement the ordinary one-step test metrics.

### 4.5 Reproducibility Setup

The benchmark is organised as a Python package under `src/srbench`. Dataset loaders, framework adapters, experiment orchestration, and metric implementations are separated modules. Human-readable JSON files under `configs` specify the dataset, seed, noise level, operator set, search parameters, and output directory for each framework–dataset combination. Dedicated runner scripts execute individual experiments, while another script materialises seed-specific configurations for repeated runs.

Each completed run writes three artefacts below the framework-specific run directory, `experiments/framework_runs/`, grouped by dataset and seed: the resolved configuration, a file containing the clean and noisy equations, and a metrics file. The metrics file also records the split and preprocessing conditions, framework metadata, the Python version, operating-system platform, and installed versions of relevant packages. Raw public data are cached under `data/raw`; their UCI source URL, DOI, domain, and licence are encoded in the dataset specification and copied to the result record. Aggregation and plotting are performed from these recorded run artefacts rather than from manually transcribed values.

Most experiments run in the project’s main virtual environment, whose direct dependencies are declared in `pyproject.toml`. The incompatible DSR and NeSymRe stacks are isolated in separate micromamba environments. Setup notes document the Python 3.7/TensorFlow 1 requirement for DSR and the PyTorch, checkpoint, and maximum-input limitations encountered for NeSymRe. Automated tests cover predictive, structure, robustness, and rollout calculations. This arrangement makes successful runs reproducible while also preserving negative evidence such as dependency failures, numerical warnings, or unavailable framework combinations.

### 4.6 Interpretability Rubric

Interpretability is treated as a multi-part assessment rather than equated with accuracy. First, equation size, tree depth, and operator set provide reproducible syntactic evidence. Second, equations are converted, where possible, from standardised variables back to the raw feature scale and simplified with SymPy. For qualitative review, representative clean-run equations are selected per dataset by sorting first by higher test  $R^2$ , then lower RMSE, and finally shorter textual length. This prevents interpretability discussion from focusing only on unusually simple but inaccurate expressions.

## 4 Methodology

The following five-point rubric guides manual inspection:

| Score | Interpretation                                                                          |
|-------|-----------------------------------------------------------------------------------------|
| 1     | The expression is too large, numerically unstable, or opaque to inspect meaningfully.   |
| 2     | Individual parts are readable, but the complete expression has no clear interpretation. |
| 3     | The expression can be understood with effort and moderate algebraic simplification.     |
| 4     | The expression is compact, readable, and mostly plausible in the dataset’s domain.      |
| 5     | The expression is simple, stable, and has a clear domain-plausible interpretation.      |

The rubric deliberately distinguishes readability from physical validity. A small equation may earn a high readability score but still omit important variables or violate domain expectations; conversely, an accurate expression may be too deep or fragile to serve as an explanatory model. Domain plausibility is therefore discussed cautiously for empirical datasets without a known law. Exact algebraic agreement can be assessed for the product and logistic-map controls, but for public data the analysis is limited to structural simplicity, stability across seeds, sensible variable use, and the predictive evidence. The rubric remains a qualitative review instrument; the current result schema records the underlying structural measures and equations but does not generate an automatic interpretability score.

# 5 Experiments

This chapter describes the concrete experiments that were executed using the protocol from Chapter 4. The emphasis is on the composition of the experiment matrix, the configurations supplied to each framework, and the extent to which the intended comparisons were completed. Numerical outcomes are reserved for Chapter 6.

## 5.1 Experimental Matrix and Task Roles

The final primary matrix combines six frameworks with nine datasets. The matrix was not designed as a collection of interchangeable regression tasks; each dataset has a specific diagnostic role. The product task checks whether an adapter can receive named variables, learn multiplication, return a valid expression, and reproduce predictions through the shared interface. The logistic map is the principal known-equation experiment. In addition to recovering an algebraic relation, it tests recursive use of the learned model. These two tasks expose implementation failures that would otherwise be difficult to distinguish from genuine difficulty on empirical data.

The seven public tasks form the surrogate-modelling part of the matrix. Airfoil Self-Noise and Yacht Hydrodynamics provide moderately sized engineering relations with five and six inputs. Energy Efficiency and Combined Cycle Power Plant provide comparatively clean, low-dimensional energy and simulation tasks. Superconductivity is a harder material-science problem in which a compact expression must approximate a relationship derived from composition statistics. Naval Propulsion tests a wider, 16-variable sensor state with a target that is highly amenable to polynomial modelling. Gas Turbine Emissions provides a ten-variable industrial-process task. Together, these tasks vary domain, input dimension, target scale, and expected equation complexity while remaining computationally feasible.

The experiment matrix uses the feature subsets and sample counts given in Table ?? . Public data are randomly sampled rather than using the complete source datasets. This choice makes repeated evolutionary searches feasible and keeps the same fixed sample count across frameworks. It also means that the experiments evaluate data-efficient symbolic surrogates, not the best accuracy obtainable by training each method on every available row.

## 5.2 Framework Configurations

Table 5.1 summarises the configurations used in the experiment files. Exact values vary by dataset because a configuration that is sufficient for the product task is not a sensible budget for a 16-variable empirical task. No broad hyperparameter optimisation was performed. Instead, small feasibility runs were used to select settings that produced valid equations within the available thesis compute budget.

Table 5.1: Main framework configurations used in the experiments.

| Framework | Search representation                 | Principal settings                                                                                                                                          |
|-----------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PySR      | Evolutionary expression trees         | 20–80 iterations, 6–12 populations of 25–30 expressions, maximum size 8–20; deterministic serial execution and best-model selection                         |
| Operon    | Multi-objective genetic programming   | Population 300–1000, 60–200 generations, 30,000–200,000 evaluations, maximum length 12–34 and depth 4–9; one thread                                         |
| GPLearn   | Tree-based genetic programming        | Population 1,000–5,000, 20–50 generations, tournament size 20, initial depth 2–6, crossover probability 0.9 and parsimony penalty                           |
| PySINDy   | Sparse polynomial library             | Polynomial degree 2 for public tasks and degree 3 for the logistic map, interaction and bias terms, STLSQ with dataset-specific threshold and ridge penalty |
| DSR       | Recurrent neural expression generator | 2,000 samples for empirical tasks and 10,000 for synthetic tasks, batch size 500, expression length 4–20, one layer with 32 hidden units                    |
| NeSymRe   | Pretrained neural generator           | Public 10M checkpoint on CPU, beam size 8 for synthetic tasks and 2 for public tasks, followed by ten BFGS constant-optimisation restarts                   |

All PySR configurations use the binary operators  $+$ ,  $-$ ,  $\times$ , and protected division, without unary functions. The population and expression limits are largest for the logistic map, where exact recovery is the objective, and slightly smaller for the public tasks. PySR’s deterministic option is enabled, parallelism is set to serial, and no extra worker process is requested.

Operon uses the corresponding symbols for addition, subtraction, multiplication,



## 5 Experiments

division, constants, and variables. Its optimisation is multi-objective with  $R^2$  and expression length as objectives. From the resulting Pareto front, the adapter selects the expression with the lowest training mean squared error. Constants are locally optimised for 20 iterations on most tasks and 30 iterations on the logistic map. The maximum number of evaluations, rather than generation count alone, acts as its principal search cap.

GPLearn uses half-and-half tree initialisation, tournament selection, and the same four arithmetic functions. Across configurations, 90% of genetic operations are crossover; subtree, hoist, and point mutation each have probability 1%. Most tasks optimise mean squared error. Naval Propulsion is the exception and uses mean absolute error because of its small target range. Dataset-specific constant ranges and parsimony coefficients are retained in the resolved configuration files.

PySINDy serves as a sparse polynomial baseline rather than a derivative estimator for dynamical systems. The adapter expands the supervised features into a polynomial library and uses sequentially thresholded least squares (STLSQ). The known product uses degree two and the logistic map degree three, which contains the necessary  $rx$  and  $rx^2$  terms. Public tasks use a quadratic library with pairwise interactions. Thresholds and ridge penalties are smaller on the synthetic tasks and stronger on the wider material-science and Naval Propulsion tasks.

DSR uses a one-layer recurrent policy with 32 hidden units, learning rate  $5 \times 10^{-4}$ , entropy weight 0.005, risk parameter  $\epsilon = 0.05$ , protected operators, and early stopping. Adding an explicit constant token to its function set was necessary for reliable recovery of the synthetic equations. NeSymRe does not train a new symbolic generator in these experiments. It loads the public pretrained checkpoint, generates candidate structures, and refines constants using BFGS. The integrated checkpoint supports at most three input variables. Therefore, on public datasets with more than three features, NeSymRe receives only the first three configured variables and the omitted variables are recorded in run metadata. Its empirical results are consequently diagnostic and not a full-feature comparison with the other frameworks.

### 5.3 Execution Procedure and Environments

The main experiment environment contains PySR 1.5.10, PyOperon 0.6.1, GPLearn 0.4.3, and PySINDy 2.1.0. The command-line runner reads one JSON configuration, constructs the requested clean and noisy datasets, instantiates the matching adapter, and executes both fits. After each fit, predictions, metrics, the equation, and framework metadata are collected. The resolved configuration, equations, and complete metrics record are then written below `experiments/framework_runs/`. Keeping the clean and noisy fit in one experiment record ensures that the robustness comparison uses the same split and seed.

## 5 Experiments

The neural frameworks could not be installed in the main environment without dependency conflicts. DSR was executed through a dedicated micromamba environment containing Python 3.7, DSR 1.0.dev0, and TensorFlow 1.15.5. NeSymRe was executed in a second environment containing NeSymRe 0.1.0 and PyTorch 1.13.1 on the CPU. Both environments use the same project source tree and runner scripts. The public tabular loader was kept compatible with Python 3.7, and `openpyxl` was installed in both neural environments for the Excel-backed energy datasets.

For each run, fitting is performed first on the clean training target and then on its noisy counterpart. Each call returns a separate fitted estimator, so the clean estimator is not reused for the noisy prediction. Test prediction is timed separately after training. In the logistic-map experiment, both fitted models additionally perform the same 50-step rollout. The common aggregation scripts subsequently read the JSON artefacts, rather than re-running models, to create per-dataset tables, repeated-seed summaries, plots, and equation reports.

### 5.4 Repeated Runs and Matrix Coverage

Stochastic search makes a single seed insufficient for estimating the typical behaviour of evolutionary SR. Repeated runs were therefore generated with seeds 7, 21, and 42. Table 5.2 reports the completed coverage used by the analysis. A “three-seed” entry means that all three seeds are present for most established tasks; exceptions are stated explicitly.

Table 5.2: Completed run coverage in the primary experiment matrix.

| Framework | Seed coverage | Qualification                                                                                                                 |
|-----------|---------------|-------------------------------------------------------------------------------------------------------------------------------|
| PySR      | 7, 21, 42     | Three seeds for all established tasks; seed 42 only for the two replacement energy datasets                                   |
| Operon    | 7, 21, 42     | Three seeds for all established tasks; seed 42 only for the two replacement energy datasets                                   |
| GPLearn   | 7, 21, 42     | Three seeds for established public tasks and the product task; logistic map and replacement energy datasets have seed 42 only |
| PySINDy   | 7, 21, 42     | Three seeds for all established tasks; seed 42 only for the two replacement energy datasets                                   |
| DSR       | 42            | One run per primary dataset in the dedicated legacy environment                                                               |

## 5 Experiments

| Framework | Seed coverage | Qualification                                                                         |
|-----------|---------------|---------------------------------------------------------------------------------------|
| NeSymRe   | 42            | One checkpoint-inference run per primary dataset; wider tasks use only three features |

The established public tasks in this table are Airfoil Self-Noise, Yacht Hydrodynamics, Superconductivity, Naval Propulsion, and Gas Turbine Emissions. Energy Efficiency and Combined Cycle Power Plant were introduced after the initial matrix as replacements for Appliances Energy and currently have only seed-42 runs. Summary tables always include `n_seeds`; means over three runs are therefore not conflated with single-run values. The asymmetry also prevents strong statistical claims about DSR, NeSymRe, or the replacement tasks.

### 5.5 Target-Noise Experiments

Robustness experiments were executed for every configured framework–dataset run. The perturbed model receives the same training inputs as the clean model, but its target is changed to  $y + \epsilon$  using the dataset-specific Gaussian noise level from Table ???. The random generator is initialised with the experiment seed, so the perturbed target can be recreated. Both models are evaluated on the identical clean test target.

The chosen noise levels reflect the different target units and scales. They range from 0.005 for the naval compressor decay coefficient to 5.0 for gas turbine nitrogen-oxide emissions. They are not intended to represent an equal percentage signal-to-noise ratio across all datasets. Accordingly, the robustness analysis is primarily within a dataset: it asks how much a given framework’s test RMSE and  $R^2$  change when the training observations are perturbed. A lower error in a noisy run is possible because SR is stochastic and the perturbation can alter the search trajectory; such cases are retained rather than forced to a non-negative degradation.

### 5.6 Logistic-Map Rollout Experiment

Only the logistic map supplies a justified iterative stability experiment. For each seed, 64 new cases are sampled from the same  $x$  and  $r$  ranges as the one-step data. The true trajectory is generated from Equation ?? for 50 steps. Starting from the same initial value, each learned expression predicts  $x_{t+1}$  from  $(x_t, r)$ , and the prediction becomes the next input. Values are clipped to  $[0, 1]$  after each step to keep the rollout within the defined state range.

## 5 Experiments

Rollouts are produced for both the clean-trained and noise-trained equations. The experiment stores RMSE and MAE at every horizon step as well as the final RMSE, mean RMSE over all predicted steps, and maximum absolute error. No rollout is attempted for the public tabular datasets because their rows are independent observations rather than ordered states of a known dynamical system. The original outline’s generic “rollout stability experiments” is therefore narrowed to this single defensible experiment.

### 5.7 Diagnostic and Excluded Experiments

Several experiments informed the final design without entering the primary framework ranking. Appliances Energy was initially run with eight temperature, humidity, wind, and pressure variables. Its poor results were investigated with a mean predictor, linear regression, quadratic ridge regression, random forest, and histogram gradient boosting. Since none produced a substantial improvement over the mean predictor on the chosen split, the SR results could not be interpreted as a clean comparison of framework capability. The task was retained as a legacy diagnostic and replaced by Energy Efficiency and Combined Cycle Power Plant in the primary matrix.

The same non-SR diagnostics were run for the remaining datasets to separate framework failure from a generally unpredictable target. These models are not black-box competitors in the primary benchmark; they are a dataset-quality check. Their role is to show whether poor SR performance occurs even though a simple polynomial or flexible tree ensemble can learn the selected feature–target relation.

An experimental NeSymRe 100M checkpoint was also tested on the logistic map and the two replacement energy datasets. It did not improve on the 10M checkpoint and produced numerical warnings during BFGS refinement. These runs are stored under a separate framework variant and excluded by default from aggregation. AI-Feynman and Deep Symbolic Optimization were investigated during framework setup, but platform and dependency constraints prevented them from becoming part of the completed matrix. Documenting these exclusions is necessary for the practical-usability comparison, but no predictive ranking is inferred from an experiment that could not be executed consistently.

## 6 Results

This chapter reports the outcomes of the experiments described in Chapter 5. Unless stated otherwise, results for PySR, Operon, GPLearn, and PySINDy on the five established public datasets are means over seeds 7, 21, and 42. Results for Energy Efficiency and Combined Cycle Power Plant are seed-42 results only. DSR and NeSymRe values are also single seed-42 runs. Standard deviations are therefore shown only where repeated evidence exists. The product task is used as an implementation check and is not included in framework rankings.

### 6.1 Known-Equation Recovery

All six adapters recovered the product  $y = ab$  to numerical precision. PySR, GPLearn, and DSR returned the direct product, PySINDy retained only the  $ab$  term from its quadratic library, Operon returned a numerically equivalent scaled product, and NeSymRe returned  $1.000000005ab$ . Clean-test  $R^2$  was 1.0 for every framework and the largest clean RMSE was below  $10^{-7}$ . The sanity check therefore confirmed that every adapter could fit, export, parse, and evaluate at least a simple multiplicative expression.

The logistic map produced a clearer separation. Table 6.1 reports the clean one-step and recursive rollout results. PySR, GPLearn, and PySINDy recovered

$$rx(1 - x) \tag{6.1}$$

exactly or in an algebraically equivalent form. Their one-step test RMSE was at machine precision, and their mean 50-step rollout error was at most  $5.4 \times 10^{-9}$ . DSR recovered  $xr(0.99999994 - 0.99999990x)$ ; its one-step RMSE was  $3.6 \times 10^{-8}$ , but repeated use amplified the small coefficient errors to a mean rollout RMSE of 0.0217. Operon also achieved essentially perfect one-step prediction, while its mean rollout RMSE was 0.0266 across three seeds.

NeSymRe was the only framework that did not recover the logistic relation. Its seed-42 expression contained a sine term, achieved only  $R^2 = 0.063$ , and had a mean rollout RMSE of 0.507. This failure occurred even though the task has only two variables and is within the checkpoint’s input limit.

## 6 Results

Table 6.1: Clean logistic-map results. Repeated values are means;  $n$  is the number of completed seeds.

| Framework | $n$ | RMSE                   | $R^2$ | Rollout RMSE           |
|-----------|-----|------------------------|-------|------------------------|
| PySR      | 3   | $2.78 \times 10^{-17}$ | 1.000 | $1.33 \times 10^{-9}$  |
| Operon    | 3   | $7.25 \times 10^{-8}$  | 1.000 | 0.0266                 |
| GPLearn   | 1   | $8.04 \times 10^{-17}$ | 1.000 | $1.96 \times 10^{-10}$ |
| PySINDy   | 3   | $4.58 \times 10^{-16}$ | 1.000 | $5.38 \times 10^{-9}$  |
| DSR       | 1   | $3.60 \times 10^{-8}$  | 1.000 | 0.0217                 |
| NeSymRe   | 1   | 0.196                  | 0.063 | 0.507                  |

## 6.2 Predictive Performance on Empirical Tasks

Table 6.2 gives clean-test  $R^2$  for the seven public tasks. Operon provided the strongest general-purpose performance. It had the highest repeated mean on Airfoil Self-Noise ( $R^2 = 0.610$ ), Yacht Hydrodynamics ( $R^2 = 0.992$ ), Superconductivity ( $R^2 = 0.451$ ), and Gas Turbine Emissions ( $R^2 = 0.807$ ). On the seed-42 Combined Cycle Power Plant run, Operon also had the highest value,  $R^2 = 0.942$ , narrowly ahead of PySINDy at 0.941.

Table 6.2: Clean-test  $R^2$  on empirical datasets. Values for DSR and NeSymRe, and all values on the two energy replacement tasks, are single seed-42 runs; the other values are three-seed means.

| Dataset                    |  | PySR  | Operon       | GPLearn | PySINDy      | DSR      | NeSymRe   |
|----------------------------|--|-------|--------------|---------|--------------|----------|-----------|
| Airfoil Self-Noise         |  | 0.419 | <b>0.610</b> | 0.491   | 0.607        | -274.516 | -0.005    |
| Yacht Hydrodynamics        |  | 0.975 | <b>0.992</b> | 0.990   | 0.910        | -0.055   | -0.083    |
| Energy Efficiency          |  | 0.907 | 0.983        | 0.920   | <b>0.994</b> | 0.845    | 0.793     |
| Combined Cycle Power Plant |  | 0.902 | <b>0.942</b> | 0.935   | 0.941        | 0.917    | -4734.276 |
| Superconductor data        |  | 0.374 | <b>0.451</b> | 0.422   | 0.265        | -0.848   | 0.155     |
| Naval Propulsion           |  | 0.084 | 0.898        | -1.009  | <b>0.999</b> | -3.002   | -0.006    |
| Gas Turbine Emissions      |  | 0.500 | <b>0.807</b> | 0.456   | 0.467        | -33.007  | -0.016    |

PySINDy was strongest when the target was well represented by its polynomial library. It obtained  $R^2 = 0.994$  and RMSE 0.761 on Energy Efficiency, and a three-seed mean  $R^2 = 0.999$  with RMSE  $5.14 \times 10^{-4}$  on Naval Propulsion. The latter result agrees with the dataset diagnostic: quadratic ridge regression achieved  $R^2 = 0.99996$

## 6 Results

on the same feature–target relation. Its performance was less competitive on Yacht Hydrodynamics, Superconductivity, and the repeated Gas Turbine runs.

PySR was consistently successful on Yacht Hydrodynamics, with mean  $R^2 = 0.975$ , but did not reach Operon on the other established empirical tasks. Its expressions produced mean  $R^2$  values of 0.419 for Airfoil, 0.374 for Superconductivity, 0.084 for Naval Propulsion, and 0.500 for Gas Turbine Emissions. GPLearn closely matched Operon on Yacht Hydrodynamics (0.990), but its variability was larger on Airfoil and Gas Turbine Emissions. On Naval Propulsion, all three GPLearn clean runs collapsed to a one-node constant expression, producing mean  $R^2 = -1.009$ .

The two neural frameworks produced mixed results. DSR exactly solved the synthetic tasks and gave useful single-run fits on Energy Efficiency ( $R^2 = 0.845$ ) and Combined Cycle Power Plant ( $R^2 = 0.917$ ). It failed badly on Airfoil, Superconductivity, Naval Propulsion, and Gas Turbine Emissions. The public NeSymRe checkpoint obtained  $R^2 = 0.793$  on Energy Efficiency but was near or below mean prediction on most other tasks. Its Combined Cycle result was numerically extreme ( $R^2 = -4734.276$ ). These empirical values must be read together with its restriction to the first three features.

The dataset baselines help contextualise the SR values. Airfoil’s best diagnostic model reached  $R^2 = 0.762$ , above the best SR mean of 0.610. Yacht’s random forest reached 0.991, essentially equal to Operon and GPLearn. On Superconductivity, histogram gradient boosting reached 0.669, showing that the target was learnable even though compact SR models reached only 0.451. On Gas Turbine Emissions, Operon’s mean 0.807 exceeded the seed-42 diagnostic baselines, whose best value was 0.766. These checks show that the observed framework gaps cannot all be attributed to intrinsically unpredictable data.

### 6.3 Computational Efficiency and Practical Execution

Training-time comparisons reflect the configured budgets rather than equal wall-clock limits. Nevertheless, clear differences appeared. Across the five repeated public tasks, Operon’s mean training times ranged from 0.275 to 0.816 seconds. PySINDy’s corresponding range was 0.005–0.103 seconds. PySR was much slower but highly consistent, requiring approximately 15.6–16.9 seconds. GPLLearn was the slowest maintainable framework, ranging from 24.0 seconds on Gas Turbine Emissions to 47.4 seconds on Yacht Hydrodynamics.

The single-run neural times depended strongly on the task. DSR required about 2.25–3.16 seconds on most established public datasets, 7.19 seconds on Energy Efficiency, and 20.83 seconds on Combined Cycle Power Plant. NeSymRe required approximately 5.0–21.2 seconds on public tasks and 30.1 seconds on the logistic map. These times exclude the one-off cost of creating the separate environments

and downloading the NeSymRe checkpoint.

Prediction was fast for every exported expression, but adapter overhead was visible. Operon and the neural adapters generally predicted complete test sets within fractions of a millisecond. PySINDy prediction was typically a few milliseconds, depending on polynomial-library width. PySR’s prediction calls were commonly around 0.01 seconds. Since the test sets contain only 100–2048 samples, these timings should be treated as small-batch measurements, not throughput benchmarks.

Practical setup separated the frameworks more strongly than prediction time. PySR, Operon, GPLEarn, and PySINDy ran in one maintained environment and completed repeated batches through a shared command. DSR required an isolated legacy TensorFlow 1 stack on Python 3.7. NeSymRe required a separate PyTorch environment, a downloaded checkpoint, and compatibility patches, while still being limited to three variables. AI-Feynman and Deep Symbolic Optimization did not reach the completed result matrix because of platform or dependency constraints. Thus, successful model execution alone understates the setup cost of the neural and legacy candidates.

## 6.4 Equation Complexity and Interpretability Evidence

Equation size varied considerably among frameworks. PySR generally returned the smallest useful expressions: across the repeated public tasks, its mean size ranged from 5 nodes on Superconductivity to 16.3 on Airfoil. Operon’s means ranged from 37.3 to 41.7 nodes. GPLEarn expressions ranged from a one-node constant on the failed Naval task to 87.3 nodes on Airfoil; its Yacht size also varied strongly between seeds. The fixed quadratic feature library made PySINDy the largest representation on wider problems, reaching 178 nodes on Superconductivity and approximately 240 and 262 nodes on Naval Propulsion and Gas Turbine Emissions. DSR and NeSymRe returned small trees, but their small size often coincided with poor prediction rather than a favourable accuracy–complexity trade-off.

The representative equations illustrate why node count alone is insufficient. On Yacht Hydrodynamics, the seed-42 PySR equation simplified to

$$\hat{R} = F_n(F_n + 0.2730)(5.2268F_n + 8.4768), \quad (6.2)$$

where  $F_n$  is the standardised Froude-number feature. It used one variable and achieved  $R^2 = 0.976$ . The seed-42 GPLEarn expression was also dominated by the Froude number and reached  $R^2 = 0.988$ , while Operon’s more complex equation used additional hull variables and reached  $R^2 = 0.993$ . This forms a concrete accuracy–simplicity trade-off rather than a single unambiguous winner.



## 6 Results

On Superconductivity, PySR produced the five-node linear relation

$$\hat{T}_c = 20.4171, n_{\text{elements}} + 35.7553, \quad (6.3)$$

with seed-42  $R^2 = 0.404$ . Operon’s larger multivariable expression reached 0.507 on the same seed. The compact PySR form is readily inspectable, but it leaves substantial predictable variation unexplained. On Naval Propulsion, PySINDy’s near-perfect fit required a roughly 240-node quadratic expression, whereas Operon’s approximately 41-node mean representation was less accurate. Consequently, the best predictive model was not necessarily the easiest equation to inspect.

Raw-feature substitution further increased textual length because every standardised variable was replaced with  $(x - \mu)/s$ . The simplified equation reports preserve both the standardised and raw-scale versions. No automatic five-point interpretability score was generated, so this chapter reports only observable evidence: equation size, depth, variable use, stability, and representative forms. Domain plausibility and the practical value of these trade-offs are considered in Chapter 7.

### 6.5 Noise Robustness and Seed Stability

The target-noise experiments did not yield a uniform framework ordering. Several noisy runs improved slightly because the stochastic search followed a different trajectory. For Airfoil, mean  $R^2$  changes were small for Operon ( $\Delta R^2 = -0.035$ ), GPLLearn ( $-0.060$ ), and PySINDy ( $-0.002$ ), where a negative value denotes an improvement. PySR decreased by 0.053. Yacht was stable for all four repeatedly evaluated frameworks, with absolute mean  $R^2$  changes below 0.005. Superconductivity was also stable for PySR, GPLLearn, and PySINDy, while Operon’s mean decrease was 0.014.

Naval Propulsion showed method-dependent sensitivity. PySINDy remained highly accurate but lost a mean 0.040 in  $R^2$  under target noise. Operon’s noisy fits improved by 0.044 on average. PySR decreased by 0.022, and GPLLearn remained at its failed constant solution. The largest instabilities occurred on Gas Turbine Emissions. Operon’s mean  $R^2$  decrease was 1.689 and GPLLearn’s was 3.389, both with very large standard deviations caused by individual noisy-run collapses. PySR’s mean decrease was 0.102. PySINDy improved on average, but its clean and noisy values were themselves highly variable across seeds.

Clean-run seed variability reinforces this distinction. Yacht was stable: the standard deviation of clean  $R^2$  was 0.001 for Operon, 0.002 for GPLLearn, PySINDy, and PySR. Airfoil standard deviations ranged from 0.019 for PySINDy to 0.215 for GPLLearn. Superconductivity ranged from 0.027 for PySR to 0.099 for PySINDy. Gas Turbine Emissions was the least stable established task, with standard deviations of 0.056 for Operon, 0.100 for PySR, 0.480 for GPLLearn, and 0.616 for PySINDy.

## 6 Results

For the logistic map, exact clean equations did not guarantee equal robustness after noisy fitting. PySR retained almost exact behaviour across three seeds, with mean  $R^2$  decrease  $1.36 \times 10^{-5}$ . PySINDy’s mean decrease was  $2.05 \times 10^{-4}$  and Operon’s was 0.00127. At seed 42, GPLearn rediscovered the exact equation from noisy observations, whereas the slightly perturbed DSR equation accumulated rollout error. NeSymRe degraded from  $R^2 = 0.063$  to  $-0.201$ . These results show that one-step noise robustness and recursive stability are related but distinct outcomes.

## **7 Discussion**

### **7.1 Interpretation of Framework Trade-Offs**

### **7.2 Suitability for Surrogate-Modelling Tasks**

### **7.3 Practical Framework Selection**

### **7.4 Limitations and Threats to Validity**

### **7.5 Research Gaps and Future Benchmarking Needs**

## **8 Conclusion**

### **8.1 Summary of Findings**

### **8.2 Answer to the Research Question**

### **8.3 Future Work**

# Bibliography

- [1] Aldeia, G. S. I., Zhang, H., Bomarito, G., Cranmer, M., Fonseca, A., Burlacu, B., La Cava, W. G. and de França, F. O. [2025], Call for action: Towards the next generation of symbolic regression benchmark, *in* ‘Proceedings of the Genetic and Evolutionary Computation Conference Companion’, Association for Computing Machinery, New York, NY, USA, pp. 1–10.  
**URL:** <https://doi.org/10.1145/3712255.3734309>
- [2] Alizadeh, R., Allen, J. K. and Mistree, F. [2020], ‘Managing computational complexity using surrogate models: A critical review’, *Research in Engineering Design* **31**(3), 275–298.  
**URL:** <https://doi.org/10.1007/s00163-020-00336-7>
- [3] Angelis, D., Sofos, F. and Karakasidis, T. E. [2023], ‘Artificial intelligence in physical sciences: Symbolic regression trends and perspectives’, *Archives of Computational Methods in Engineering* **30**(6), 3845–3865.  
**URL:** <https://doi.org/10.1007/s11831-023-09922-z>
- [4] Brum, B. R., Lober, L., Previdelli, I. and Rodrigues, F. A. [2026], ‘Discovering equations from data: Symbolic regression in dynamical systems’, *Journal of Physics: Complexity* **7**(1), 012001.  
**URL:** <https://doi.org/10.1088/2632-072X/ae3eed>
- [5] Brunton, S. L., Proctor, J. L. and Kutz, J. N. [2016], ‘Discovering governing equations from data by sparse identification of nonlinear dynamical systems’, *Proceedings of the National Academy of Sciences* **113**(15), 3932–3937.  
**URL:** <https://doi.org/10.1073/pnas.1517384113>
- [6] Burlacu, B., Kronberger, G. and Kommenda, M. [2020], Operon C++: An efficient genetic programming framework for symbolic regression, *in* ‘Proceedings of the Genetic and Evolutionary Computation Conference Companion’, Association for Computing Machinery, New York, NY, USA, pp. 1562–1570.  
**URL:** <https://doi.org/10.1145/3377929.3398099>
- [7] Cranmer, M. [2023], ‘Interpretable machine learning for science with PySR and SymbolicRegression.jl’.  
**URL:** <https://arxiv.org/abs/2305.01582>
- [8] de França, F. O. and Aldeia, G. S. I. [2021], ‘Interaction–transformation evolutionary algorithm for symbolic regression’, *Evolutionary Computation*

## Bibliography

- 29**(3), 367–390.  
**URL:** [https://doi.org/10.1162/evco\\_a\\_00285](https://doi.org/10.1162/evco_a_00285)
- [9] de França, F. O., Virgolin, M., Kommenda, M., Majumder, M. S., Cranmer, M., Espada, G., Ingelse, L., Fonseca, A., Landajuela, M., Petersen, B. et al. [2025], ‘SRBench++: Principled benchmarking of symbolic regression with domain-expert interpretation’, *IEEE Transactions on Evolutionary Computation* **29**(4), 1127–1134.  
**URL:** <https://doi.org/10.1109/TEVC.2024.3423681>
- [10] Dong, J. and Zhong, J. [2025], ‘Recent advances in symbolic regression’, *ACM Computing Surveys* **57**(11), 1–37.  
**URL:** <https://doi.org/10.1145/3735634>
- [11] Espada, G., Ingelse, L., Canelas, P., Barbosa, P. and Fonseca, A. [2022], Data types as a more ergonomic frontend for grammar-guided genetic programming, in ‘Proceedings of the 21st ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences’, GPCE 2022, Association for Computing Machinery, pp. 86–94.  
**URL:** <https://doi.org/10.1145/3564719.3568697>
- [12] Guo, J. and Yin, W.-J. [2024], ‘Harnessing data using symbolic regression methods for discovering novel paradigms in physics’, *Science China Physics, Mechanics & Astronomy* **67**(6), 267301.  
**URL:** <https://doi.org/10.1007/s11433-023-2346-2>
- [13] La Cava, W. G., Orzechowski, P., Burlacu, B., de França, F. O., Virgolin, M., Jin, Y., Kommenda, M. and Moore, J. H. [2021], Contemporary symbolic regression methods and their relative performance, in ‘Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks’, Vol. 1.  
**URL:** <https://openreview.net/forum?id=xVQMrDLyGst>
- [14] Makke, N. and Chawla, S. [2024], ‘Interpretable scientific discovery with symbolic regression: A review’, *Artificial Intelligence Review* **57**(2).  
**URL:** <https://doi.org/10.1007/s10462-023-10622-0>
- [15] Orzechowski, P., La Cava, W. and Moore, J. H. [2018], Where are we now? a large benchmark study of recent symbolic regression methods, in ‘Proceedings of the Genetic and Evolutionary Computation Conference’, Association for Computing Machinery, New York, NY, USA, pp. 1183–1190.  
**URL:** <https://doi.org/10.1145/3205455.3205539>
- [16] Petersen, B. K., Landajuela Larma, M., Mundhenk, T. N., Santiago, C. P., Kim, S. K. and Kim, J. T. [2021], Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients, in ‘International Conference on Learning Representations’.  
**URL:** <https://arxiv.org/abs/1912.04871>

## Bibliography

- [17] Rudin, C. [2019], ‘Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead’, *Nature Machine Intelligence* **1**(5), 206–215.  
**URL:** <https://doi.org/10.1038/s42256-019-0048-x>
- [18] Sahoo, S. S., Lampert, C. H. and Martius, G. [2018], Learning equations for extrapolation and control, *in* ‘Proceedings of the 35th International Conference on Machine Learning’, Vol. 80 of *Proceedings of Machine Learning Research*, PMLR, pp. 4442–4450.  
**URL:** <https://proceedings.mlr.press/v80/sahoo18a.html>
- [19] Stephens, T. [2026], ‘gplearn: Genetic programming in python’, Software documentation. Accessed 20 August 2026.  
**URL:** <https://gplearn.readthedocs.io/en/stable/>
- [20] Udrescu, S.-M. and Tegmark, M. [2020], ‘AI Feynman: A physics-inspired method for symbolic regression’, *Science Advances* **6**(16), eaay2631.  
**URL:** <https://doi.org/10.1126/sciadv.aay2631>
- [21] Wang, G., Wang, E., Li, Z., Zhou, J. and Sun, Z. [2024], ‘Exploring the mathematic equations behind the materials science data using interpretable symbolic regression’, *Interdisciplinary Materials* **3**(4), 637–657.  
**URL:** <https://doi.org/10.1002/idm2.12180>